

Data Gateway Design Pattern

Lecture notes



Data Gateway: motivation

- An object-oriented program needs to interact with a relational database
 - Mismatch of paradigms
 - If SQL statements are scattered throughout the application
 - Can violate architectural (e.g. MVC) structure
 - Complicate code
 - Have more dependencies than needed
- 

Data Gateway: advantages

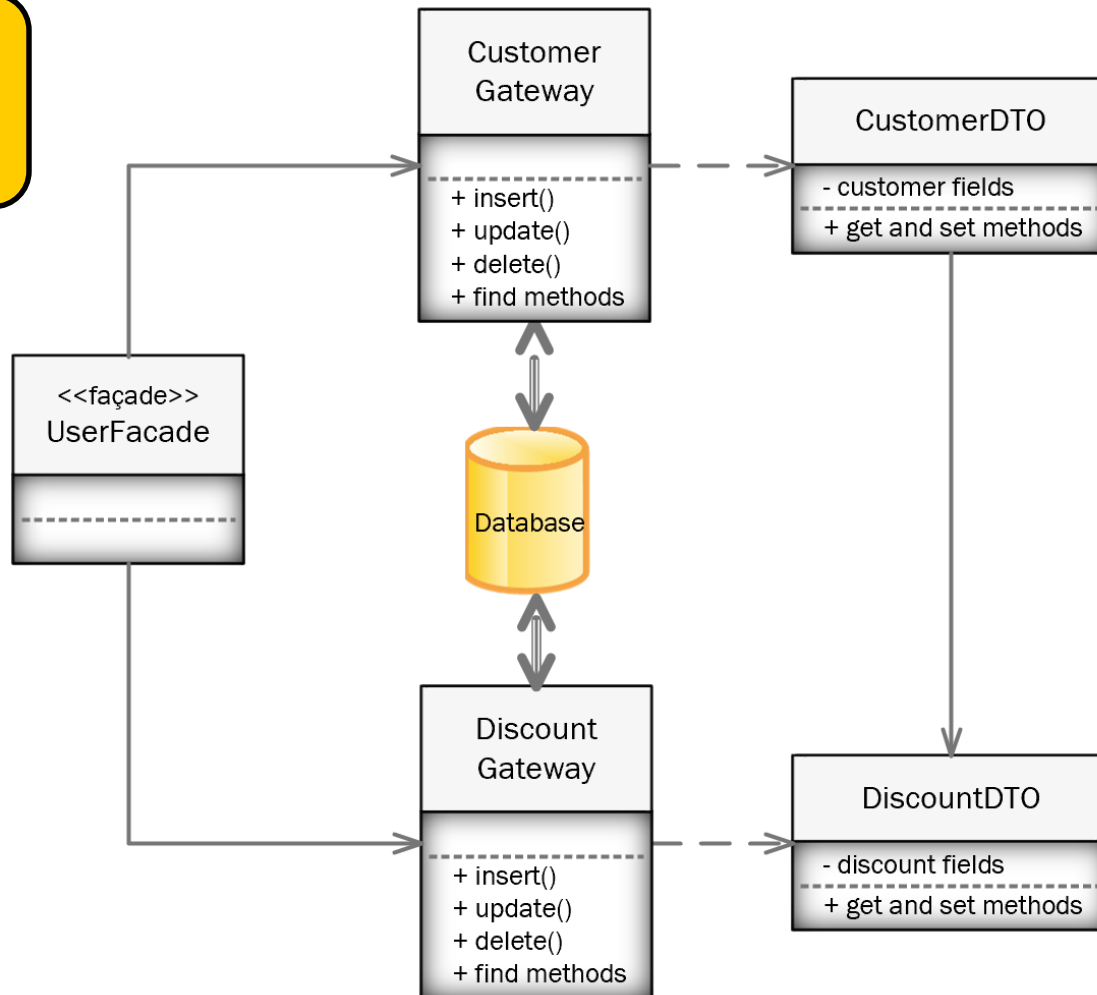
- Preserve MVC
- Separate the code that interacts with the relational database from business logic
- Centralise database interaction, including the SQL, in one or more well-defined classes
 - Easier to maintain
 - Easier to extend functionality
 - Improves the OO structure
- Focus can be
 - a single table, or
 - a view of multiple tables

Data Gateway: in practice

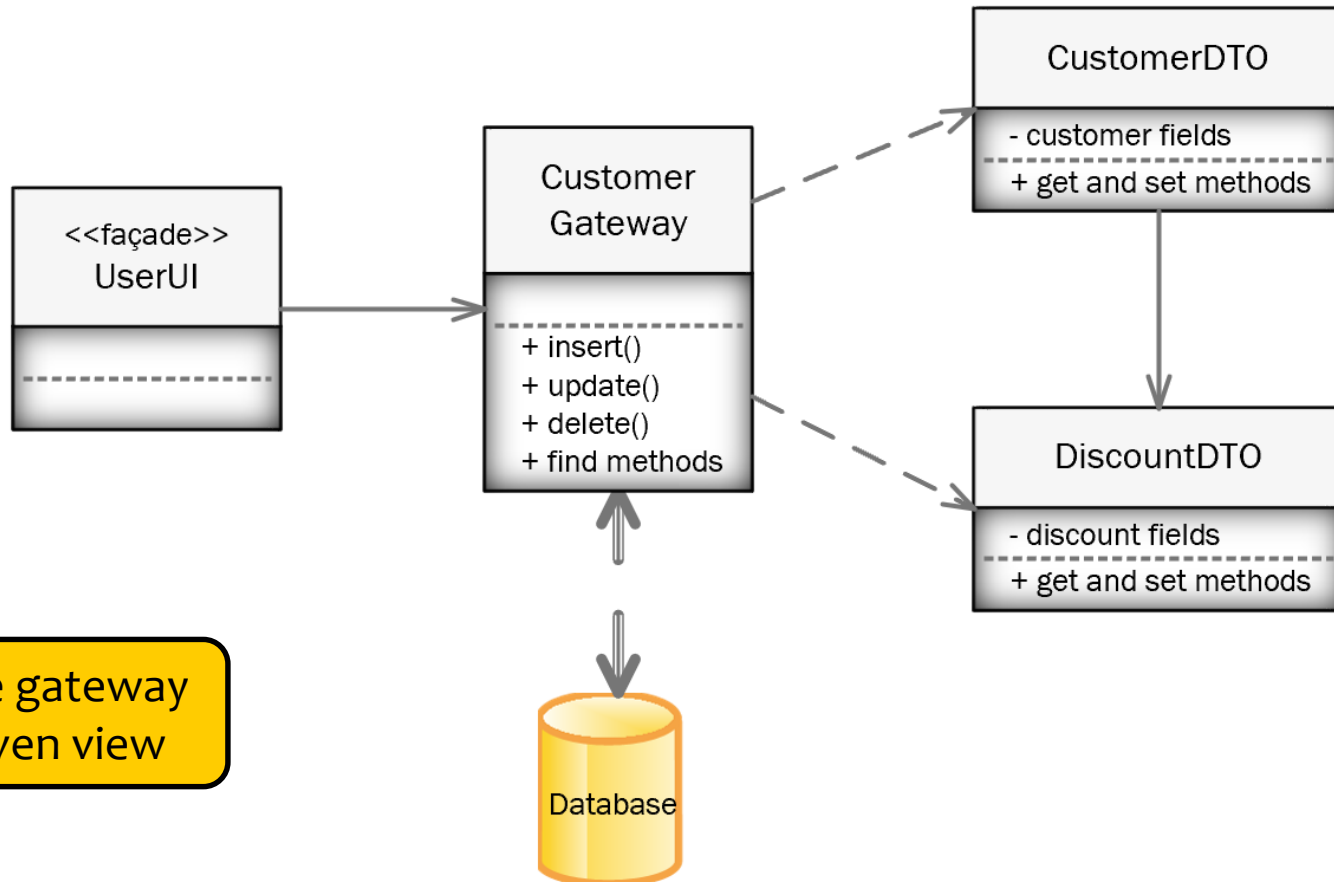
- A simple interface with several standard methods
 - `update()`
 - `insert()`
 - `delete()`
 - A series of `find()` methods for querying the database
- Holds all the SQL for accessing a single table or view
 - Essentially a wrapper class for SQL statements
 - Other classes call the gateway's methods for all interaction with the database table
 - Typically, one data gateway class per table or view

Table Data Gateway example

A separate gateway for each table



View Data Gateway example



A single gateway
for a given view

Activity



Coding

- Review the example code and watch out for:
 - how the starting point project has database commands scattered throughout the code
 - how the project with patterns has cleaner code because it uses data gateways
- If you have any questions, ask

Projects: `LectureExample_StartingPoint`
and: `LectureExample_WithPatterns`